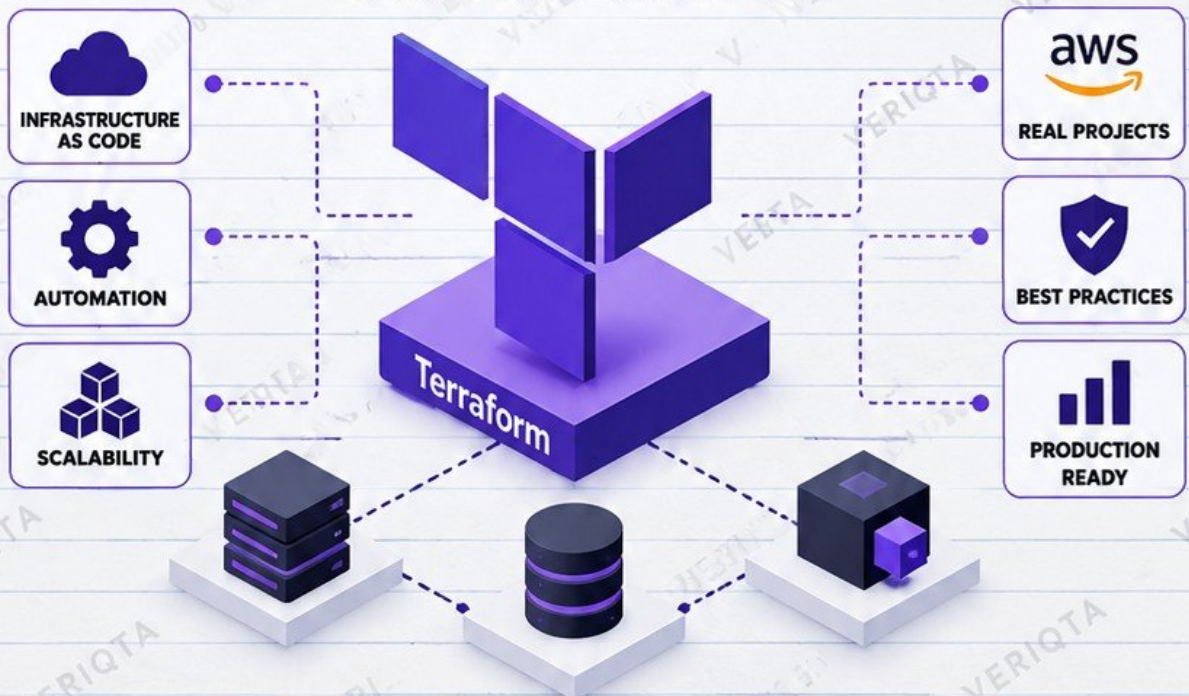


Terraform

Real-World Projects

HANDBOOK

BUILD PRODUCTION-READY INFRASTRUCTURE
ON AWS WITH TERRAFORM



- ✓ 20 HANDS-ON PROJECTS
- ✓ STEP-BY-STEP GUIDES

- ✓ REAL AWS ARCHITECTURES
- ✓ TROUBLESHOOTING SCENARIOS

- ✓ DEV TO PROD WORKFLOWS
- ✓ JUNIOR ENGINEER FRIENDLY



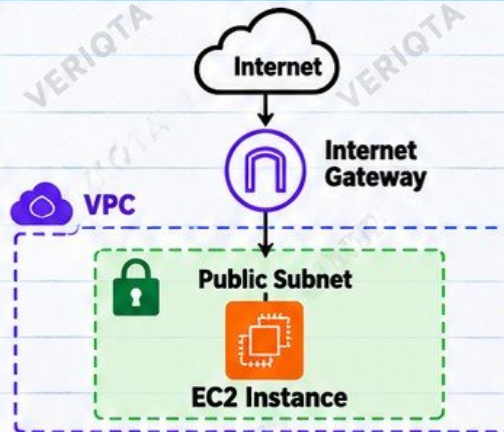
Your First Real Terraform AWS Deployment

Build a small AWS environment with a VPC, subnet, security group, and EC2 instance. Learn the complete init → plan → apply → verify → destroy workflow.

WHAT YOU WILL BUILD

- ✓ VPC (Virtual Private Cloud)
- ✓ Subnet (public)
- ✓ Internet Gateway
- ✓ Security Group
- ✓ EC2 instance (Linux)

ARCHITECTURE DIAGRAM



WHY THIS MATTERS

- You learn how AWS resources work together.
- You understand Terraform real workflow.
- You create infrastructure as code instead of using the AWS console.
- This is the foundation for bigger real-world projects like three-tier architectures.

TERRAFORM WORKFLOW



KEY COMMANDS

- | | |
|--------------------------------|---|
| <code>terraform init</code> | ➔ Download providers and initialize the working directory. |
| <code>terraform plan</code> | ➔ Show what resources will be created, changed, or destroyed. |
| <code>terraform apply</code> | ➔ Create the infrastructure in AWS. |
| <code>terraform destroy</code> | ➔ Remove all resources created by this project. |

JUNIOR ENGINEER TIP

Always run `terraform plan` before `terraform apply`. It helps you catch mistakes before creating real infrastructure and avoids unexpected costs.

COMMON BEGINNER MISTAKES

- ✗ Using the wrong AWS region
- ✗ Not configuring AWS credentials
- ✗ Forgetting to destroy resources
- ✗ Opening SSH (port 22) to 0.0.0.0/0
- ✗ Hardcoding sensitive information

PRACTICE THIS

- ✓ Create the project from scratch
- ✓ Verify the EC2 instance is running
- ✓ SSH into your instance
- ✓ Check the resources in AWS console
- ✓ Destroy everything at the end

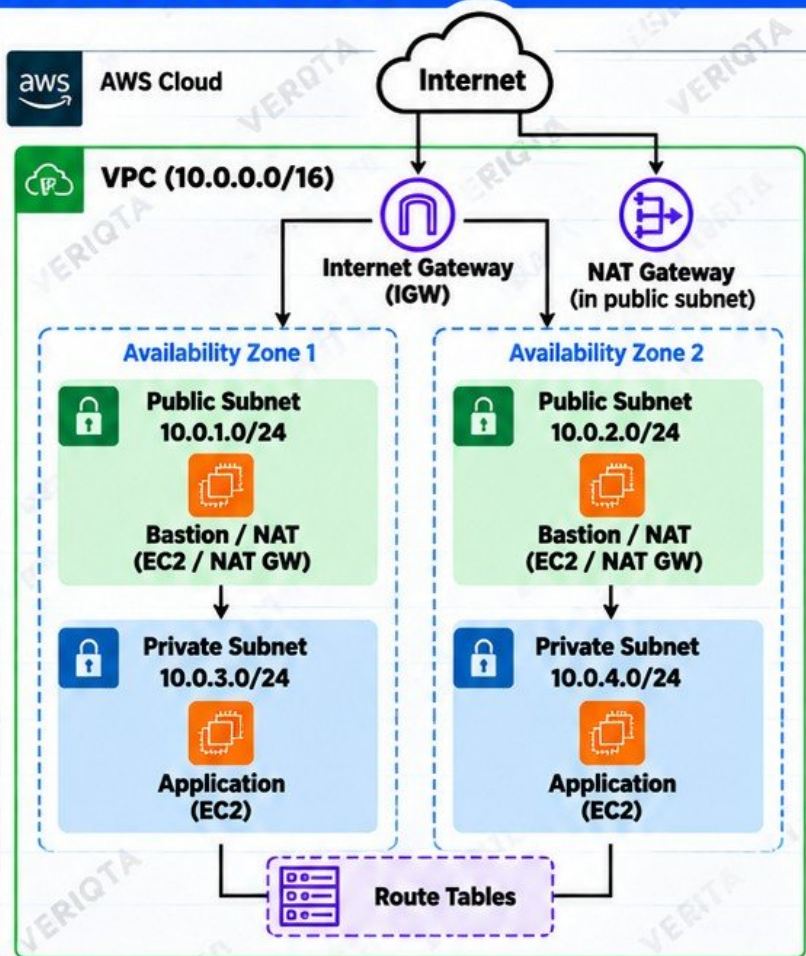
REAL WORLD CONNECTION

This is the same workflow engineers use to deploy real infrastructure in development, staging, and production environments — just on a larger scale.

Build a Production-Style VPC

Create a VPC with public and private subnets across Availability Zones, route tables, an Internet Gateway, and NAT connectivity. Understand why applications should not all live in public subnets.

ARCHITECTURE DIAGRAM



KEY VPC COMPONENTS

- VPC**
Isolated network in AWS (10.0.0.0/16).
- Subnets**
Public and private subnets across Availability Zones.
- Internet Gateway (IGW)**
Allows inbound and outbound internet access for public subnets.
- NAT Gateway**
Allows private subnets to access the internet (outbound only).
- Route Tables**
Define where network traffic goes.
- Security Groups**
Act as a virtual firewall for instances.

WHY NOT PUT EVERYTHING IN PUBLIC SUBNETS?

- Increases the attack surface.
- Exposes databases and internal services.
- Not aligned with security best practices.
- You lose network isolation and control.
- Harder to meet compliance requirements.

ROUTE TABLE EXAMPLE

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	igw-xxxxxxx (for public subnet)
0.0.0.0/0	nat-xxxxxxx (for private subnet)

USEFUL TERRAFORM COMMANDS

- `terraform init` → Initialize the project.
- `terraform plan` → Preview changes.
- `terraform apply` → Create the VPC and resources.
- `terraform destroy` → Remove all resources.
- `terraform fmt` → Format your code.
- `terraform validate` → Check for syntax errors.

JUNIOR ENGINEER TIPS

- Use at least 2 Availability Zones for high availability.
- Keep databases in private subnets.
- Use NAT Gateway instead of NAT instance.
- Apply proper tags to all resources.
- Test the VPC using a bastion host or SSM.

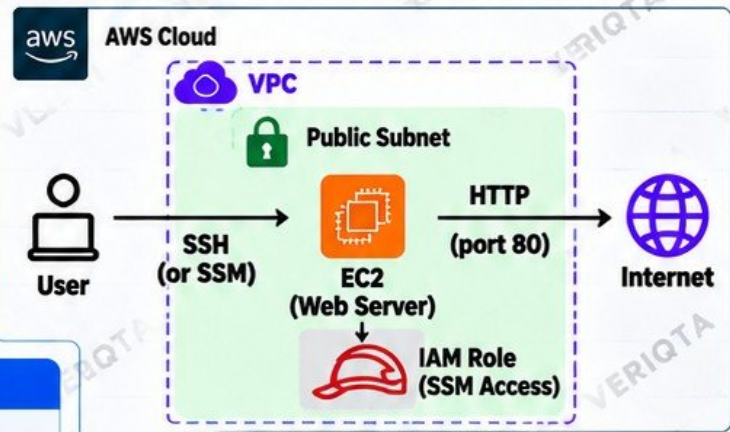
Deploy a Secure EC2 Web Server

Provision EC2 with Terraform, security groups, IAM role, user data, tags, and SSH or Systems Manager access. Avoid hardcoded credentials and unrestricted inbound access.

WHAT YOU WILL BUILD

- ✓ EC2 instance (Amazon Linux)
- ✓ Security group (allow HTTP, SSH/SSM)
- ✓ IAM role for EC2 (SSM access)
- ✓ User data to install a web server
- ✓ Tags for better resource management

ARCHITECTURE DIAGRAM



KEY TERRAFORM RESOURCES

- | | |
|----------------------------------|---|
| <code>aws_instance</code> | → Creates the EC2 instance. |
| <code>aws_security_group</code> | → Controls inbound and outbound traffic. |
| <code>aws_iam_role</code> | → Allows EC2 to access AWS services (e.g. SSM). |
| <code>aws_iam_role_policy</code> | → Attaches permissions to the role. |
| <code>user_data</code> | → Runs commands when the instance starts. |
| <code>tags</code> | → Helps with organization and cost tracking. |

USER DATA EXAMPLE (INSTALL APACHE)

```

resource "aws_instance" "web" {
  ami           = "ami-0c02fb55956c7d316" # Amazon Linux 2023
  instance_type = "t2.micro"
  subnet_id     = aws_subnet.public.id
  iam_instance_profile = aws_iam_instance_profile.ec2_profile.name
  vpc_security_group_ids = [aws_security_group.web_sg.id]

  user_data = <<-EOF
  #!/bin/bash
  yum update -y
  yum install -y httpd
  systemctl enable httpd
  systemctl start httpd
  echo "<h1>Welcome to VERIQTA!</h1>" > /var/www/html/index.html
  EOF
}

tags = {
  Name        = "veriqta-web-server"
  Environment = "dev"
  Project     = "terraform-learning"
}

```

EXAMPLE SECURITY GROUP (HTTP + SSM)

```

resource "aws_security_group" "web_sg" {
  name        = "web-sg"
  description = "Allow HTTP and SSM"
  vpc_id      = aws_vpc.main.id

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"] # HTTP (for demo)
  }

  # Allow SSM (no need to open SSH if using SSM)
  # SSH example (restrict to your IP):
  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["YOUR.IP.ADDRESS/32"]
  }
}

```

COMMON MISTAKES

- ✗ Allowing SSH from 0.0.0.0/0
- ✗ Not using an IAM role
- ✗ Hardcoding AWS credentials
- ✗ Forgetting to add tags
- ✗ Using the wrong AMI or region
- ✗ Exposing the instance unnecessarily

JUNIOR ENGINEER TIPS

- ✓ Use SSM instead of SSH for better security.
- ✓ Always use IAM roles, not access keys.
- ✓ Restrict access to only the required ports.
- ✓ Use tags for organization.
- ✓ Test the web server in a browser.
- ✓ Keep your Terraform code in a Git repository.

PRACTICE THIS

- ✓ Deploy the EC2 instance using Terraform
- ✓ Access it using SSM or SSH
- ✓ Verify the web page in a browser
- ✓ Destroy all resources

Build a Reusable EC2 Module

Refactor repeated infrastructure into a Terraform module. Introduce inputs, outputs, module structure, variable validation, and reusable infrastructure patterns.

WHY USE TERRAFORM MODULES?

- ✓ Reuse infrastructure across projects
- ✓ Keep configurations organized
- ✓ Reduce copy and paste
- ✓ Make it easier to manage and update
- ✓ Follow infrastructure as code best practices
- ✓ Use the same module for dev, staging and prod

MODULE STRUCTURE

```

modules/
├── ec2/
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   ├── terraform.tfvars.example
│   └── README.md

```

Each file has a purpose:

- main.tf → resources
- variables.tf → input variables with validation
- outputs.tf → output values
- terraform.tfvars.example → example inputs
- README.md → usage notes

MAIN.TF (EC2 RESOURCE)

```

resource "aws_instance" "this" {
  ami           = var.ami_id
  instance_type = var.instance_type
  subnet_id     = var.subnet_id
  vpc_security_group_ids = var.security_group_ids
  iam_instance_profile = var.iam_instance_profile
  user_data     = var.user_data

  tags = merge(
    { Name = var.name },
    var.tags
  )
}

```

VARIABLES.TF (INPUTS)

```

variable "ami_id" {
  description = "AMI ID for the EC2 instance"
  type        = string
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"

  validation {
    condition =
      can(regex("^[t2|t3]\\d{1,2}$", var.instance_type))
    error_message = "Use a valid T2 or T3 instance type."
  }
}

```

OUTPUTS.TF (OUTPUTS)

```

output "instance_id" {
  description = "The EC2 instance ID"
  value       = aws_instance.this.id
}

output "public_ip" {
  description = "The public IP address"
  value       = aws_instance.this.public_ip
}

```

EXAMPLE USAGE (IN ROOT MODULE)

```

module "web_server" {
  source          = "../modules/ec2"
  ami_id          = "ami-0abc123456789def0"
  instance_type   = "t3.micro"
  subnet_id       = aws_subnet.public.id
  security_group_ids = [aws_security_group.web.id]
  iam_instance_profile = aws_iam_instance_profile.ec2.id
  user_data       = file("user_data.sh")
  name            = "web-server"

  tags = {
    Environment = "dev"
    Project     = "terraform-modules"
  }
}

```

USER_DATA.SH (EXAMPLE)

```

#!/bin/bash
yum update -y
yum install -y httpd
systemctl enable httpd
systemctl start httpd
echo "<h1>Welcome to VERIQTA!</h1>" >
/var/www/html/index.html

```

KEY CONCEPTS

- ✓ Inputs (variables)
- ✓ Outputs
- ✓ Variable validation
- ✓ Reusability
- ✓ Separation of concerns
- ✓ Clean module structure
- ✓ Use tags for organization

COMMON MISTAKES

- ✗ Hardcoding values instead of using variables
- ✗ Not using variable validation
- ✗ Forgetting outputs
- ✗ Keeping everything in one file
- ✗ Not using tags
- ✗ Making the module too specific (not reusable)

JUNIOR ENGINEER TIPS

- 💡 Use meaningful variable names
- 💡 Add validation to prevent bad input
- 💡 Keep your module generic and reusable
- 💡 Document inputs and outputs
- 💡 Use the same module across environments
- 💡 Add tags for cost tracking
- 💡 Test your module before using in production

REAL WORLD USE CASES

- ✓ Standard EC2 module for web servers
- ✓ Reusable modules for entire teams
- ✓ Consistent infrastructure across environments
- ✓ Faster deployment of new projects
- ✓ Easier maintenance and updates
- ✓ Foundation for larger modules (ALB, ASG, RDS, etc.)

VERIQTA

LEARN | BUILD | GROW



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

A Better
Engineer
Every Day

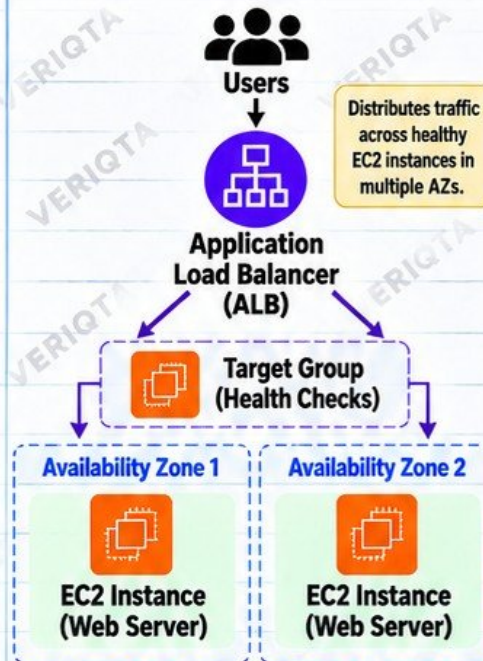
Build a Highly Available Web Tier

Deploy EC2 instances across multiple Availability Zones behind an Application Load Balancer. Configure listeners, target groups, security-group relationships, and health checks.

WHAT YOU WILL BUILD

- ✓ Application Load Balancer (ALB)
- ✓ Target Group with health checks
- ✓ EC2 instances in multiple AZs
- ✓ Security groups with correct rules
- ✓ Listener (HTTP/HTTPS)
- ✓ A highly available web tier

ARCHITECTURE DIAGRAM



WHY THIS MATTERS

- ✓ High availability across multiple Availability Zones.
- ✓ Automatically routes traffic only to healthy instances.
- ✓ Handles instance failures without downtime.
- ✓ Scales with Auto Scaling (covered in next project).
- ✓ Common design for production web applications.

KEY AWS COMPONENTS

-  **Application Load Balancer**
Layer 7 load balancing.
-  **Target Group**
Routes traffic to healthy instances.
-  **EC2 Instances**
Run your web application.
-  **Security Groups**
Control inbound and outbound traffic.

ALB (MAIN RESOURCE)

```

resource "aws_lb" "web" {
  name          = "web-alb"
  load_balancer_type = "application"
  subnets     = var.public_subnet_ids
  security_groups = [aws_security_group.alb.id]

  tags = {
    Name = "web-alb"
  }
}

```

TARGET GROUP

```

resource "aws_lb_target_group" "web" {
  name     = "web-tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = var.vpc_id

  health_check {
    path            = "/"
    protocol        = "HTTP"
    healthy_threshold = 2
    unhealthy_threshold = 2
    interval        = 30
  }
}

```

LISTENER (HTTP)

```

resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.web.arn
  port              = 80
  protocol          = "HTTP"

  default_action {
    type = "forward"
    target_group_arn = aws_lb_target_group.web.arn
  }
}

```

COMMON MISTAKES

- ✗ Putting EC2 instances in only one AZ.
- ✗ Using a load balancer with the wrong type (Use Application Load Balancer for HTTP/HTTPS).
- ✗ Not configuring health checks.
- ✗ Allowing 0.0.0.0/0 to the instances directly.
- ✗ Forgetting to attach instances to the target group.
- ✗ Using the same security group for ALB and EC2.

JUNIOR ENGINEER TIPS

- 💡 Use at least 2 Availability Zones.
- 💡 Allow inbound traffic to ALB, not directly to EC2 instances.
- 💡 Use health checks to detect unhealthy instances.
- 💡 Keep instances in private subnets (for production).
- 💡 Use meaningful tags (Name, Environment, Project).

PRACTICE THIS

- ✓ Deploy the ALB and target group.
- ✓ Launch 2 EC2 instances in different AZs.
- ✓ Attach the instances to the target group.
- ✓ Verify both instances are healthy.
- ✓ Test the load balancer DNS name in a browser.
- ✓ Stop one instance and confirm traffic still works (high availability).

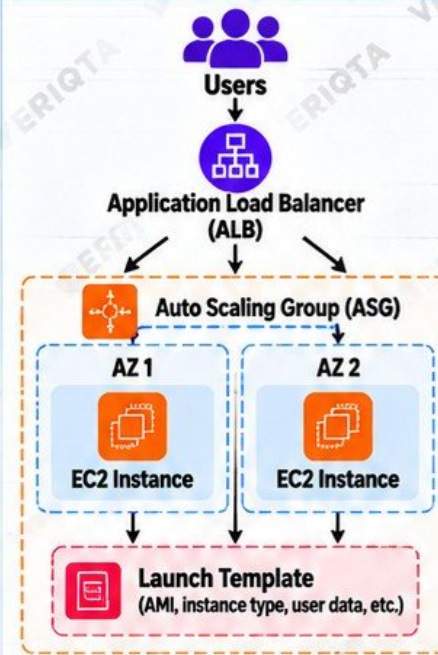
Add Auto Scaling

Build a Launch Template and Auto Scaling Group. Connect it to the load balancer and understand desired, minimum, and maximum capacity.

WHAT YOU WILL BUILD

- ✓ EC2 Launch Template
- ✓ Auto Scaling Group (ASG)
- ✓ Connect ASG to Application Load Balancer
- ✓ Configure desired, minimum and maximum capacity
- ✓ Health checks and scaling policies
- ✓ Test scaling in real scenarios

ARCHITECTURE DIAGRAM



HOW AUTO SCALING WORKS

- 1 **Launch Template**
Defines how EC2 instances should be created.
- 2 **Auto Scaling Group**
Maintains the desired number of instances across AZs.
- 3 **Load Balancer**
Distributes traffic only to healthy instances.
- 4 **Scaling Policies**
Automatically adds or removes instances based on conditions (e.g. CPU, requests).

ASG CAPACITY SETTINGS

Setting	What It Means
Minimum	Smallest number of instances to keep running.
Desired	Target number of instances (what ASG tries to maintain).
Maximum	Largest number of instances allowed during scale out.

LAUNCH TEMPLATE (EXAMPLE)

```

resource "aws_launch_template" "web" {
  name         = "web-lt"
  image_id     = "ami-0c02fb55956c7d316"
  instance_type = "t3.micro"
  user_data    = file("user_data.sh")

  tag_specifications {
    resource_type = "instance"
    tags = {
      Name      = "web-server"
      Environment = "dev"
    }
  }
}
  
```

AUTO SCALING GROUP (EXAMPLE)

```

resource "aws_autoscaling_group" "web" {
  name             = "web-asg"
  min_size         = 2
  desired_capacity = 2
  max_size         = 5
  vpc_zone_identifier = [
    var.public_subnet_ids
  ]
  target_group_arns = [
    aws_lb_target_group.web.arn
  ]
  launch_template {
    id      = aws_launch_template.web.id
    version = "$Latest"
  }
}
  
```

SCALING POLICY (EXAMPLE)

```

resource "aws_autoscaling_policy" "cpu" {
  name             = "cpu-scale"
  scaling_adjustment = 1
  adjustment_type  = "ChangeInCapacity"
  cooldown          = 300
  autoscaling_group_name = aws_autoscaling_group.web.name

  metric_aggregation_type = "Average"
}
  
```

COMMON MISTAKES

- ✗ Using a single AZ.
- ✗ Not attaching the ASG to the load balancer.
- ✗ Setting min, desired and max to the same value.
- ✗ Using an outdated AMI or instance type.
- ✗ Not configuring health checks.
- ✗ Forgetting to update the launch template version.

JUNIOR ENGINEER TIPS

- 💡 Use at least 2 Availability Zones.
- 💡 Set a realistic min, desired and max.
- 💡 Use health checks from the load balancer.
- 💡 Keep your launch template updated.
- 💡 Test scaling by generating traffic (e.g. using a load testing tool).
- 💡 Use meaningful tags (Name, Environment, Project).

PRACTICE THIS

- ✓ Create a launch template.
- ✓ Create an Auto Scaling Group.
- ✓ Attach it to your existing ALB.
- ✓ Set min = 2, desired = 2, max = 4.
- ✓ Add a CPU-based scaling policy.
- ✓ Generate traffic and watch new instances launch.
- ✓ Terminate the traffic and confirm instances scale in.

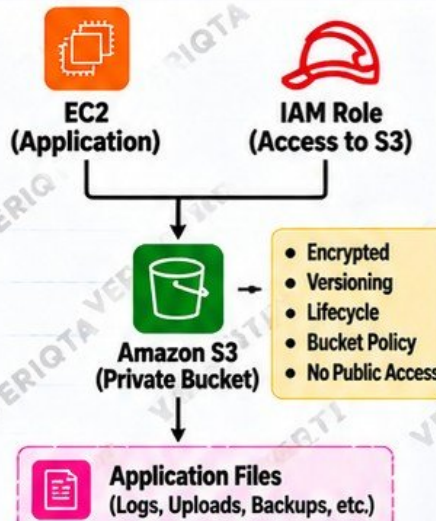
Build an S3 Application Storage Layer

Provision private S3 storage with encryption, versioning, lifecycle configuration, bucket policies, and controlled application access.

WHAT YOU WILL BUILD

- ✓ S3 bucket for application storage
- ✓ Block public access (private bucket)
- ✓ Server-side encryption (SSE-S3 or SSE-KMS)
- ✓ Versioning enabled
- ✓ Lifecycle configuration
- ✓ Bucket policy with controlled access
- ✓ IAM role for application access
- ✓ Test upload and access from an EC2 instance

ARCHITECTURE DIAGRAM



KEY S3 FEATURES

-  **Private bucket** (Block public access)
-  **Encryption** (SSE-S3 or SSE-KMS)
-  **Versioning** (Keep previous versions)
-  **Lifecycle rules** (Automate storage costs)
-  **Bucket policy** (Control who can access)
-  **IAM roles** (Grant application access)

S3 BUCKET (EXAMPLE)

```

resource "aws_s3_bucket" "app_storage" {
  bucket        = "veriqta-app-storage"
  force_destroy = false
  tags = {
    Name       = "veriqta-app-storage"
    Environment = "dev"
    Project    = "terraform-learning"
  }
}
  
```

ENCRYPTION & VERSIONING

```

resource "aws_s3_bucket_versioning" "this" {
  bucket = aws_s3_bucket.app_storage.id
  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "this" {
  bucket = aws_s3_bucket.app_storage.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}
  
```

LIFECYCLE CONFIGURATION

```

resource "aws_s3_bucket_lifecycle_configuration" "this" {
  bucket = aws_s3_bucket.app_storage.id
  rule {
    id       = "log-cleanup"
    status   = "Enabled"
    filter {
      prefix = "logs/"
    }
    expiration {
      days = 90
    }
    transition {
      days          = 30
      storage_class = "STANDARD_IA"
    }
  }
}
  
```

BUCKET POLICY (EXAMPLE)

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::123456789012:role/app-role"
      },
      "Action": ["s3:PutObject", "s3:GetObject"],
      "Resource": "arn:aws:s3:::veriqta-app-storage/*"
    }
  ]
}
  
```

COMMON MISTAKES

- ✗ Making the bucket public
- ✗ Not enabling encryption
- ✗ Not enabling versioning
- ✗ Overly permissive bucket policies
- ✗ Using IAM users instead of roles
- ✗ Forgetting lifecycle rules
- ✗ Storing sensitive data without encryption
- ✗ Not testing access from the application

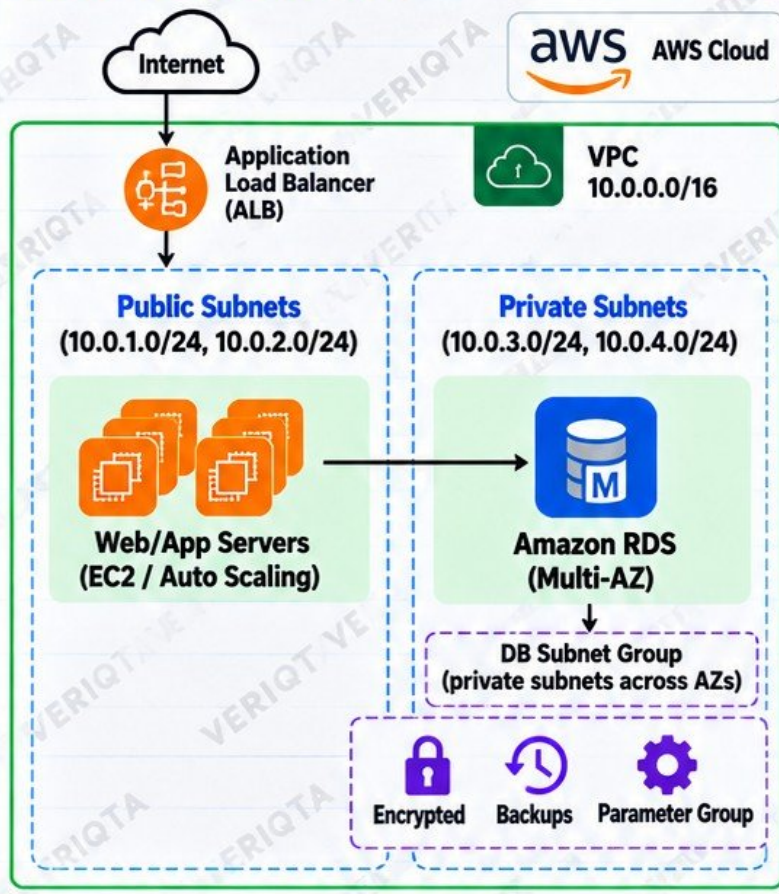
JUNIOR ENGINEER TIPS

-  Always block public access.
-  Use IAM roles, not access keys.
-  Enable versioning from day one.
-  Use lifecycle rules to reduce costs.
-  Follow the principle of least privilege.
-  Use meaningful bucket names and tags.
-  Test upload and download from your EC2 instance.
-  Monitor access using CloudTrail.

Build a Private RDS Database Layer

Deploy RDS into private subnets with a DB subnet group, security groups, encryption, backups, parameter configuration, and application-to-database connectivity.

ARCHITECTURE DIAGRAM



KEY COMPONENTS

- Amazon RDS**
Managed relational database service (e.g. MySQL, PostgreSQL, Aurora).
- DB Subnet Group**
Uses private subnets across multiple Availability Zones.
- Security Group**
Allows database traffic only from application servers (e.g. port 3306).
- Encryption**
Encrypts data at rest and in transit using AWS KMS.
- Backups**
Automated backups and optional point-in-time recovery.
- Parameter Group**
Configures database engine settings.

WHY USE PRIVATE SUBNETS?

- Keeps the database away from the internet.
- Reduces the attack surface.
- Only application servers can access it.
- Follows AWS security best practices.
- Helps you meet compliance requirements.

IMPORTANT TERRAFORM RESOURCES

<code>aws_db_instance</code>	Creates the RDS database.
<code>aws_db_subnet_group</code>	Defines the private subnets.
<code>aws_security_group</code>	Controls database access.
<code>aws_db_parameter_group</code>	Sets database parameters.
<code>aws_kms_key</code>	Manages encryption.
<code>aws_db_snapshot</code>	Creates on-demand backups.

EXAMPLE CONFIGURATION (SNIPPET)

```
resource "aws_db_instance" "main" {
  identifier = "veriqta-db"
  engine    = "mysql"
  instance_class = "db.t3.micro"
  allocated_storage = 20
  db_subnet_group_name = aws_db_subnet_group.main.name
  vpc_security_group_ids = [aws_security_group.rds.id]
  storage_encrypted = true
  kms_key_id = aws_kms_key.rds.arn
  backup_retention_period = 7
  multi_az = true
  parameter_group_name = aws_db_parameter_group.main.name
}
```

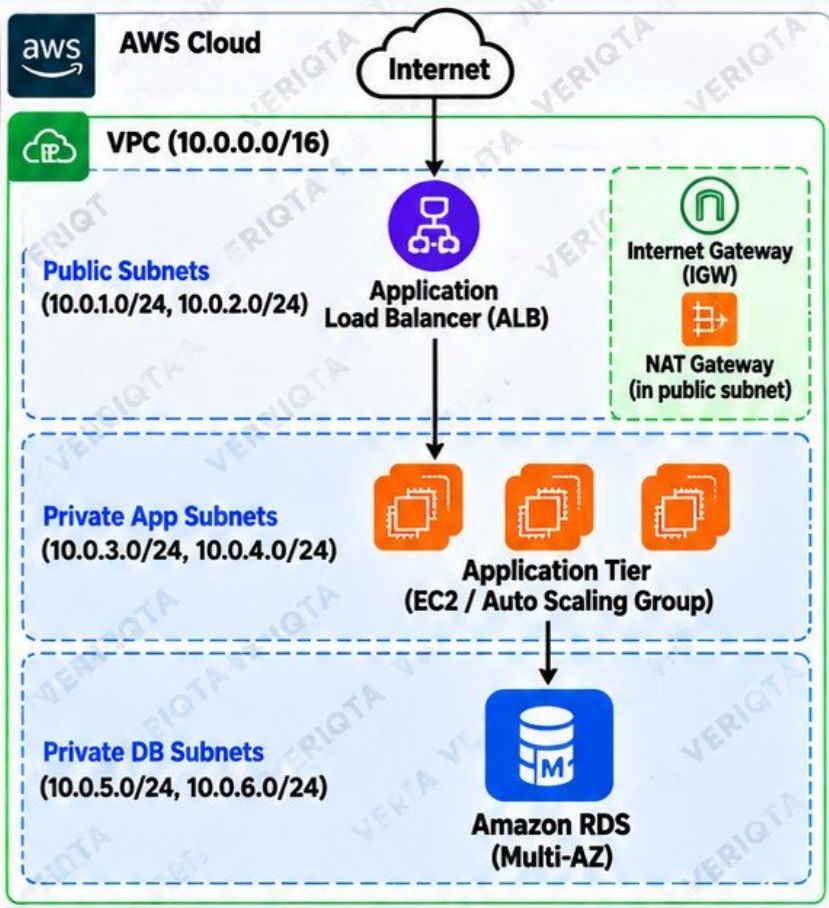
JUNIOR ENGINEER TIPS

- Always use private subnets for RDS.
- Allow access only from application security groups.
- Enable encryption and backups from the start.
- Use Multi-AZ for production.
- Choose the right instance size for your workload.
- Test connectivity from an EC2 instance in the same VPC.

Build the **AWS** Three-Tier Architecture

Combine the previous skills into a complete architecture:
Internet → **ALB** → **Application Tier** → **RDS**.
 Separate public, application, and database network boundaries.

ARCHITECTURE DIAGRAM



HOW TRAFFIC FLOWS

- 1 Users access your application through the internet.
- 2 The Application Load Balancer (ALB) receives traffic in public subnets.
- 3 Traffic is sent to the application tier in private subnets (EC2 Auto Scaling Group).
- 4 The application connects to Amazon RDS in private subnets.
- 5 Database is not publicly accessible.

KEY BENEFITS

- ✓ Improved security with network isolation
- ✓ Scalable and highly available
- ✓ Follows AWS best practices
- ✓ Separate public, application, and database layers
- ✓ Ready for real-world applications

WHY NOT PUT EVERYTHING IN PUBLIC SUBNETS?

- ✗ Increases the attack surface
- ✗ Exposes databases to the internet
- ✗ Does not follow security best practices
- ✗ Harder to meet compliance requirements
- ✗ Higher risk of data breaches

CORE AWS RESOURCES USED

-  **VPC**
Isolated network
-  **Subnets**
Public and private subnets
-  **Internet Gateway (IGW)**
Provides internet access
-  **Application Load Balancer**
Distributes traffic to EC2
-  **EC2 + Auto Scaling**
Application tier
-  **Amazon RDS**
Managed database
-  **NAT Gateway**
Outbound internet for private subnets
-  **Security Groups**
Control network access

TERRAFORM CONCEPTS

- VPC, subnets, route tables
- Security groups
- ALB, target groups, listeners
- Launch template & Auto Scaling
- RDS and DB subnet group
- Proper resource dependencies
- Variables and outputs

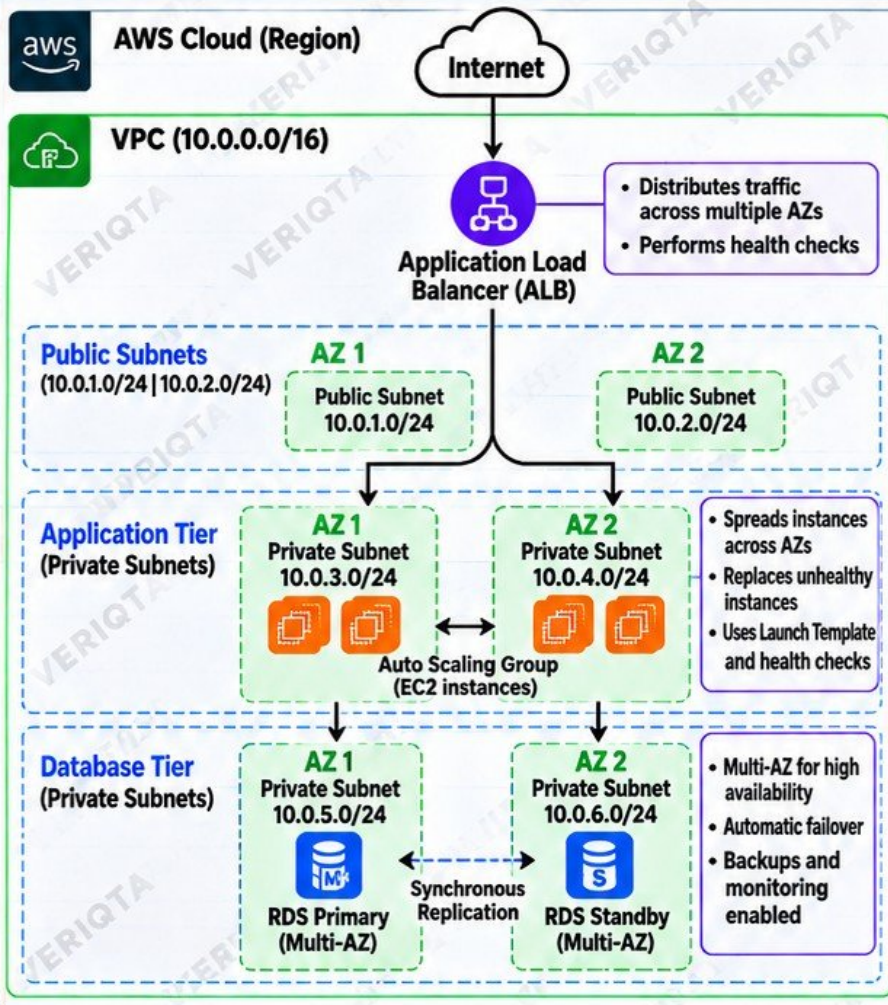
JUNIOR ENGINEER TIPS

-  Keep your database in private subnets.
-  Allow only application security groups to access RDS.
-  Use multiple Availability Zones for high availability.
-  Test the application end-to-end.
-  Use clear tags for all resources.

Make the Three-Tier Architecture Highly Available

Expand the architecture across Availability Zones. Add Auto Scaling, Multi-AZ database configuration, health checks, failure boundaries, and resilient networking.

HIGHLY AVAILABLE THREE-TIER ARCHITECTURE



KEY HIGH AVAILABILITY FEATURES

- ✓ Multi-AZ deployment
- ✓ Application Load Balancer
- ✓ Auto Scaling Group
- ✓ RDS Multi-AZ
- ✓ Health checks
- ✓ Resilient networking
- ✓ Proper subnet design
- ✓ Fault isolation
- ✓ Automatic recovery

FAILURE SCENARIOS HANDLED

- ⚡ EC2 instance failure
- ⚡ Availability Zone outage
- ⚡ Unhealthy application instance
- ⚡ Database failover
- ⚡ Network issues
- ⚡ Increased traffic (scaling)
- ⚡ Maintenance events

WHY IT MATTERS

- 🎯 Keeps your application available even when components fail.
- Reduces downtime and data loss.
- Meets production and business requirements.
- Follows AWS best practices.

IMPORTANT AWS RESOURCES

-  **VPC**
Networking
-  **RDS (Multi-AZ)**
Managed database
-  **ALB**
Load balancing
-  **Route Tables**
Traffic routing
-  **EC2 + Auto Scaling**
Application tier
-  **NAT Gateway**
Outbound internet
-  **Security Groups**
Access control

USEFUL TERRAFORM CONCEPTS

- VPC, subnets, route tables
- ALB, target groups, listeners
- Launch template & Auto Scaling
- RDS with Multi-AZ
- Security groups and NACLs
- Health checks and tags
- Modular and reusable code

JUNIOR ENGINEER TIPS

- 💡 Use at least 2 Availability Zones.
- 💡 Keep application and database in private subnets.
- 💡 Use Multi-AZ for RDS.
- 💡 Configure health checks for ALB and EC2.
- 💡 Test failure scenarios.
- 💡 Use meaningful tags for all resources.
- 💡 Monitor your infrastructure.

VERIQTA

LEARN | BUILD | GROW



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

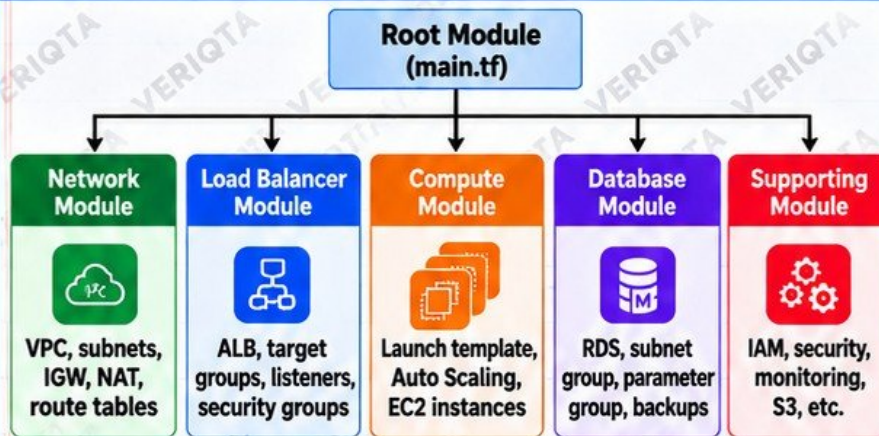
*A Better
Engineer
Every Day*



Refactor the Architecture into Terraform Modules

Break the growing infrastructure into network, compute, load-balancer, database, and supporting modules. Learn module boundaries and dependency management.

MODULAR ARCHITECTURE OVERVIEW



Modules work together to create a complete, reusable infrastructure.

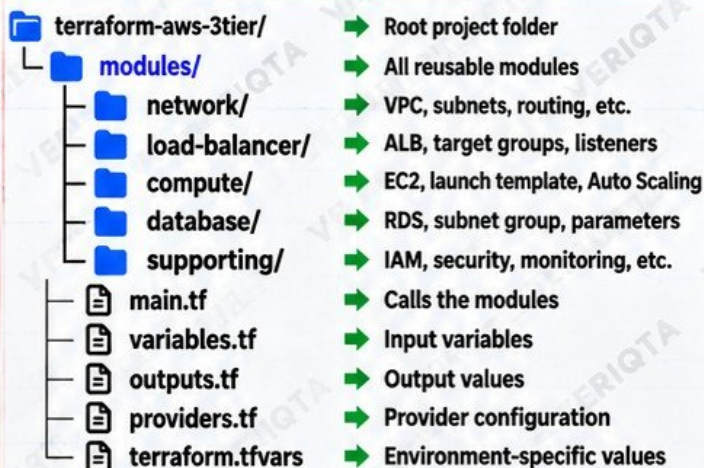
KEY BENEFITS

- ✓ Easier to manage and maintain
- ✓ Reusable infrastructure
- ✓ Clear separation of responsibilities
- ✓ Scales for multi-environment setups (dev, staging, prod)
- ✓ Reduces code duplication
- ✓ Better collaboration in teams
- ✓ Supports testing and versioning

MODULE BOUNDARIES

- Each module has a clear purpose
- Modules should be independent
- Use inputs (variables) and outputs
- Avoid tightly coupled modules
- Keep modules focused and small
- Use consistent naming and structure

EXAMPLE TERRAFORM PROJECT STRUCTURE



MODULE USAGE EXAMPLE (main.tf)

```

module "vpc" {
  source = "../modules/network"
  vpc_cidr = "10.0.0.0/16"
  env = "dev"
}

module "alb" {
  source = "../modules/load-balancer"
  vpc_id = module.vpc.vpc_id
  public_subnets = module.vpc.public_subnet_ids
}

module "app" {
  source = "../modules/compute"
  subnet_ids = module.vpc.private_subnet_ids
  alb_target_group_arn = module.alb.target_group_arn
}

module "db" {
  source = "../modules/database"
  subnet_ids = module.vpc.private_subnet_ids
  db_name = "veriqta"
}

```

JUNIOR ENGINEER TIPS

- 💡 Start with small, focused modules.
- 💡 Use clear input and output variables.
- 💡 Test each module independently.
- 💡 Document what each module does.
- 💡 Use Terraform outputs to pass values between modules.
- 💡 Keep module interfaces simple and consistent.

COMMON MISTAKES

- ✗ Putting everything in one module
- ✗ Poor module naming
- ✗ Not using outputs between modules
- ✗ Tight coupling between modules
- ✗ Hardcoding values inside modules
- ✗ Ignoring module versioning

REAL WORLD CONNECTION

- 🎯 This modular structure is how real engineering teams build and maintain production infrastructure using Terraform. It makes the architecture scalable, reusable, and easier to manage across different environments.



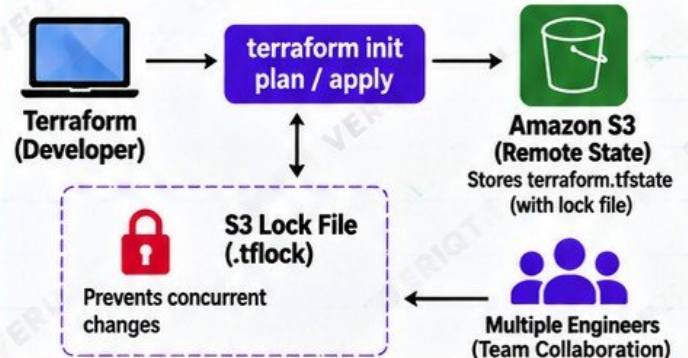
Move Terraform State to a Remote Backend

Move local state to an S3 backend and introduce state locking using the current supported S3 backend locking approach. Explain why shared teams must not pass terraform.tfstate files around.

WHY USE A REMOTE BACKEND?

- ✓ Keeps state files safe and centralized
- ✓ Enables team collaboration
- ✓ Provides state locking (prevents concurrent changes)
- ✓ Reduces the risk of state file loss or corruption
- ✓ Supports multiple environments (dev, staging, prod)
- ✓ Improves security, access control, and auditability

ARCHITECTURE DIAGRAM



S3 BACKEND CONFIGURATION (EXAMPLE)

```
terraform {
  backend "s3" {
    bucket = "veriqta-terraform-state"
    key    = "three-tier/prod/terraform.tfstate"
    region = "us-east-1"
    encrypt = true
    use_lockfile = true
  }
}
```

Annotations:

- `bucket = "veriqta-terraform-state"` → S3 bucket name
- `key = "three-tier/prod/terraform.tfstate"` → Path to state file
- `region = "us-east-1"` → AWS region
- `encrypt = true` → Encrypt state at rest
- `use_lockfile = true` → Enable S3 lock file (current recommendation)

INITIALIZE AND VERIFY

```
# Initialize with S3 backend
terraform init

# Check backend configuration
terraform init -reconfigure

# View current state information
terraform state list

# Apply your changes
terraform plan
terraform apply
```

REQUIRED S3 BUCKET SETTINGS

-  **Enable versioning**
(keeps history and allows recovery)
-  **Enable encryption**
(SSE-S3 or SSE-KMS)
-  **Use a bucket policy**
(restrict access to your team/IPs)
-  **Enable access logging**
(optional but recommended)

IAM PERMISSIONS (MINIMUM)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::veriqta-terraform-state",
        "arn:aws:s3:::veriqta-terraform-state/*"
      ]
    }
  ]
}
```

WHY NOT SHARE terraform.tfstate?

- ✗ It contains your entire infrastructure and sensitive information.
- ✗ Manual sharing causes version conflicts and resource drift.
- ✗ It can be accidentally modified or lost.
- ✗ It bypasses state locking.
- ✗ It is not secure and not auditable.
- ✗ It does not scale for teams.

JUNIOR ENGINEER TIPS

- 💡 Always use a remote backend from day one.
- 💡 Enable versioning and encryption on S3.
- 💡 Use `use_lockfile = true` (no DynamoDB required).
- 💡 Give least-privilege IAM permissions.
- 💡 Keep separate state files for each environment.
- 💡 Never commit terraform.tfstate to Git.

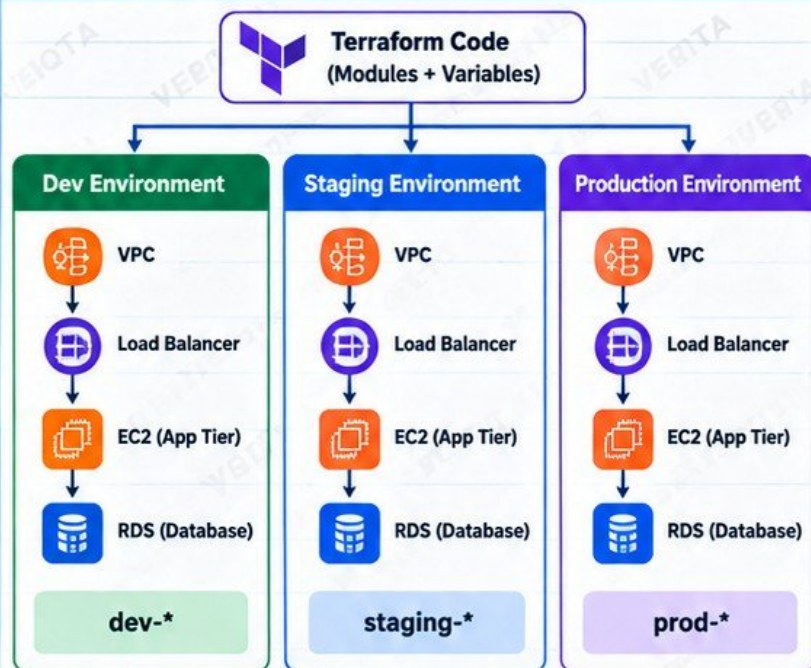
USEFUL COMMANDS

<code>terraform init</code>	Initialize backend
<code>terraform plan</code>	Preview changes
<code>terraform apply</code>	Apply changes
<code>terraform state list</code>	List resources
<code>terraform state show <resource></code>	View details
<code>terraform init -migrate-state</code>	Move from local to S3

Build Dev, Staging, and Production Environments

Turn one architecture into separate environments. Manage environment-specific variables, state isolation, naming, sizing, and configuration without copy-and-paste infrastructure.

THE BIG PICTURE



KEY CONCEPTS

- Environment Variables**
Store values like region, instance size, and domain per environment.
- State Isolation**
Use separate state files or workspaces for each environment.
- Naming Conventions**
Use clear, consistent names (e.g. app-dev, app-staging, app-prod).
- Sizing**
Adjust instance types and resources for each environment.
- Configuration**
Use variables and tfvars files instead of copy-paste code.

COMMON MISTAKES

- ✗ Using the same state for all environments.
- ✗ Hardcoding values in the code.
- ✗ Copy-pasting resources instead of modules.
- ✗ Inconsistent naming across environments.

ENVIRONMENT-SPECIFIC EXAMPLES

DEV

- ✓ Smaller instance sizes
- ✓ Fewer resources
- ✓ Lower cost
- ✓ Used for testing and development

Example Variables (dev.tfvars)

```
environment = "dev"
instance_type = "t3.micro"
db_instance_class = "db.t3.micro"
domain_name = "dev.example.com"
```

STAGING

- ✓ Similar to production
- ✓ Slightly larger resources
- ✓ Used for final testing
- ✓ Mirrors real user traffic

Example Variables (staging.tfvars)

```
environment = "staging"
instance_type = "t3.small"
db_instance_class = "db.t3.small"
domain_name = "staging.example.com"
```

PRODUCTION

- ✓ Largest and most stable
- ✓ High availability
- ✓ Monitoring and backups
- ✓ Used by real users

Example Variables (prod.tfvars)

```
environment = "prod"
instance_type = "t3.medium"
db_instance_class = "db.t3.medium"
domain_name = "example.com"
```

JUNIOR ENGINEER TIP

Use Terraform workspaces or separate state files for each environment. This keeps your environments isolated and prevents accidental changes.

PRACTICE THIS

- 1 Create dev, staging, and prod tfvars files.
- 2 Use variables in your main configuration.
- 3 Deploy the same module to all environments.
- 4 Verify each environment is isolated.

DEVOPS CONNECTION

Multiple environments help you test safely, reduce risk, and make deployments predictable. This is a core practice in real DevOps and platform engineering teams.

Deploy Infrastructure Through GitHub Actions



Create a Terraform CI workflow that runs formatting, initialization, validation, and planning on proposed changes. Add controlled deployment instead of engineers applying everything manually from laptops.

HOW IT WORKS



WHY USE GITHUB ACTIONS?

- ✓ Automates Terraform workflow
- ✓ Catches errors early
- ✓ Ensures consistent deployments
- ✓ Reduces manual mistakes
- ✓ Keeps an audit trail
- ✓ Supports team collaboration
- ✓ Uses environment protection rules
- ✓ Safer than running from laptops

EXAMPLE WORKFLOW FILE

```

.github/workflows/terraform.yml

name: Terraform CI                                # Trigger on push and pull request
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
jobs:
  terraform:
    runs-on: ubuntu-latest                         # Run on Ubuntu runner
    steps:
      - name: Checkout code                         # Get the latest code
        uses: actions/checkout@v4
      - name: Setup Terraform                       # Install Terraform
        uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: 1.7.0
      - name: Terraform Format                       # Check code formatting
        run: terraform fmt -check
      - name: Terraform Init                         # Initialize the project
        run: terraform init
      - name: Terraform Validate                     # Validate the configuration
        run: terraform validate
      - name: Terraform Plan                         # Create an execution plan
        run: terraform plan -out=tfplan
  
```

OPTIONAL APPLY JOB (AFTER MERGE)

```

- name: Terraform Apply
  if: github.ref == 'refs/heads/main'
  run: terraform apply -auto-approve tfplan
  
```

ENVIRONMENT PROTECTION (BEST PRACTICE)

- 🔒 Use GitHub Environments (dev, staging, prod)
- 👥 Require approval for production deployments
- 🔑 Use OIDC with AWS (no long-lived credentials)
- 📄 Set environment-specific variables and secrets

COMMON MISTAKES

- ✗ Running terraform apply on every pull request
- ✗ Storing AWS credentials in code
- ✗ Not using environment protection rules
- ✗ Skipping terraform validate
- ✗ Using the same workspace for all environments

KEY COMMANDS (LOCAL REFERENCE)

```

terraform fmt    ➡ Format configuration files
terraform init   ➡ Initialize the project
terraform validate ➡ Check for errors
terraform plan   ➡ Show what will change
terraform apply  ➡ Create or update infrastructure
  
```

JUNIOR ENGINEER TIP

💡 Use CI for validation and planning, and restrict apply to approved changes only. This keeps your infrastructure safe, consistent, and auditable.

REAL-WORLD CONNECTION

🎯 This is the same workflow used by real DevOps and platform engineering teams to deploy infrastructure in development, staging, and production.

Add Infrastructure Security Controls



+



Harden IAM, security groups, encryption, secrets handling, state protection, provider configuration, and network exposure. Identify dangerous Terraform patterns before deployment.

KEY SECURITY AREAS

-  **IAM**
Use least privilege. Avoid using root. Use IAM roles instead of access keys.
-  **Security Groups**
Allow only required traffic. Avoid 0.0.0.0/0 unless necessary.
-  **Encryption**
Enable encryption for data at rest and in transit (RDS, S3, EBS, etc).
-  **Secrets Handling**
Use AWS Secrets Manager or SSM Parameter Store.
-  **State Protection**
Store state in S3 with lockfile. Restrict access and enable encryption.
-  **Provider Configuration**
Use IAM roles or OIDC. Avoid hardcoded credentials.
-  **Network Exposure**
Keep databases and internal services in private subnets.

EXAMPLE: LEAST PRIVILEGE IAM ROLE

```
resource "aws_iam_role" "ec2_role" {
  name = "veriqta-ec2-role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17",
    Statement = [{
      Effect = "Allow",
      Principal = { Service = "ec2.amazonaws.com" },
      Action = "sts:AssumeRole"
    }]
  })
}
```

Allows EC2 to assume this role. Attach only the permissions you need.

EXAMPLE: SECURE S3 BACKEND (STATE)

```
terraform {
  backend "s3" {
    bucket = "veriqta-terraform-state"
    key    = "prod/terraform.tfstate"
    region = "us-east-1"
    encrypt = true
    use_lockfile = true
  }
}
```

Stores state in S3 with encryption and native lockfile.

EXAMPLE: SECURITY GROUP (RESTRICTED)

```
resource "aws_security_group" "app_sg" {
  name = "app-sg"
  vpc_id = aws_vpc.main.id

  ingress {
    from_port = 80
    to_port   = 80
    protocol = "tcp"
    cidr_blocks = [aws_vpc.main.cidr_block] # ALB or VPC only
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

Allow traffic only from trusted sources. Avoid 0.0.0.0/0 for inbound rules.

DANGEROUS PATTERNS TO AVOID

-  Hardcoded AWS access keys in code or variables.
-  Using broad permissions (e.g. AdministratorAccess).
-  Public databases (RDS) or open security groups.
-  Storing secrets in Terraform state or plain text.
-  Using 0.0.0.0/0 for SSH or database access.
-  Using local state in team projects.
-  Disabling encryption.
-  Ignoring provider version constraints.

BEST PRACTICES

-  Use least privilege IAM roles.
-  Restrict security group access.
-  Enable encryption by default.
-  Use Secrets Manager or Parameter Store.
-  Store Terraform state in S3 with encryption and lockfile.
-  Use private subnets for internal resources.
-  Set explicit provider versions and required versions.
-  Tag all resources.
-  Scan configuration before deployment (e.g. Checkov).

JUNIOR ENGINEER TIP



Security is not an afterthought. Build it into your infrastructure from the beginning. It is much easier and cheaper to prevent security issues than to fix them in production.

REAL-WORLD IMPACT



Secure infrastructure reduces risk, protects sensitive data, and keeps your environments compliant. These practices are used by real DevOps and platform engineering teams in production.

VERIQTA

LEARN | BUILD | GROW



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

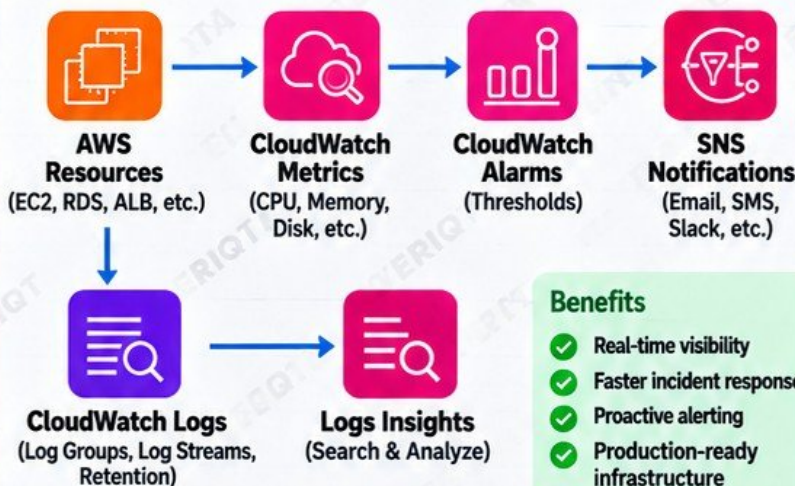
A Better
Engineer
Every Day

Add Monitoring and Operational Visibility



Provision CloudWatch alarms, log-related infrastructure, SNS notifications, and useful operational signals with Terraform. Connect infrastructure creation to production observability.

ARCHITECTURE DIAGRAM



KEY COMPONENTS

-  **CloudWatch Metrics**
Collects metrics from AWS resources (e.g. EC2, RDS, ALB).
-  **CloudWatch Alarms**
Triggers alerts based on thresholds (e.g. high CPU, low disk space).
-  **SNS Notifications**
Sends alerts via email, SMS, or integrations (Slack, Teams, etc.).
-  **CloudWatch Logs**
Stores application and system logs with retention policies.
-  **CloudWatch Logs Insights**
Query and analyze logs for troubleshooting.

EXAMPLE: CLOUDWATCH ALARM WITH SNS (TERRAFORM)

```

resource "aws_cloudwatch_metric_alarm" "ec2_cpu" {
  alarm_name          = "veriqta-ec2-cpu-high"
  comparison_operator = "GreaterThanOrEqualToThreshold"
  evaluation_periods   = 2
  metric_name         = "CPUUtilization"
  namespace           = "AWS/EC2"
  period              = 300
  statistic            = "Average"
  threshold            = 80
  alarm_description   = "Triggers when EC2 CPU is above 80%"
  alarm_actions       = [aws_sns_topic.alerts.arn]
  dimensions = {
    InstanceId = aws_instance.web.id
  }
}
  
```

This alarm triggers when EC2 CPU is above 80% and sends a notification via SNS.

COMMON ALARMS TO CREATE

- ✓ EC2 CPU utilization
- ✓ EC2 status check failures
- ✓ Low disk space on EC2 (via CloudWatch Agent)
- ✓ RDS CPU, free storage, and connection count
- ✓ ALB target unhealthy hosts
- ✓ High 4xx/5xx errors (via ALB access logs)
- ✓ Application errors in logs

COMMON MISTAKES

- ✗ No alarms configured
- ✗ Using default thresholds
- ✗ Not setting log retention (unlimited logs)
- ✗ No notifications for critical alerts
- ✗ Ignoring application-level metrics
- ✗ Creating infrastructure without observability
- ✗ Not testing alerts

JUNIOR ENGINEER TIPS

- 💡 Start with basic alarms and expand over time.
- ✓ Use meaningful alarm names and tags.
- ✓ Set log retention to control costs.
- ✓ Test your SNS notifications.
- ✓ Include monitoring in every environment (dev, staging, prod).

USEFUL TERRAFORM RESOURCES

- 🔗 `aws_cloudwatch_metric_alarm`
- 🔗 `aws_cloudwatch_log_group`
- 🔗 `aws_cloudwatch_log_stream`
- 🔗 `aws_sns_topic`
- 🔗 `aws_sns_topic_subscription`

REAL-WORLD IMPACT

- 🎯 Detects issues before users are affected.
- ✓ Reduces downtime and MTTR.
- ✓ Provides visibility into system health.
- ✓ Helps with capacity planning.
- ✓ Essential for production environments.

Build a Container Infrastructure Project

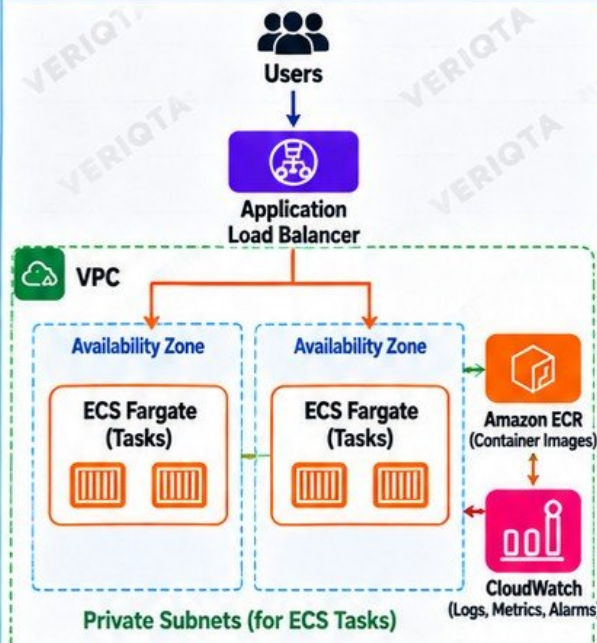


Provision the AWS infrastructure required for containerized workloads, such as VPC networking, IAM, load balancing, ECR, and ECS Fargate. Understand how Terraform supports application platforms.

KEY SERVICES

-  **VPC**
Network foundation for containerized workloads.
-  **IAM**
Secure access for ECS and other AWS services.
-  **ECR**
Store and manage container images.
-  **ECS Fargate**
Run containers without managing servers.
-  **Application Load Balancer**
Distribute traffic to containerized applications.
-  **CloudWatch**
Monitor containers and infrastructure.

ARCHITECTURE DIAGRAM



PROJECT GOALS

- ✓ Build a production-ready container infrastructure.
- ✓ Use Terraform to provision all required AWS resources.
- ✓ Understand how containers fit into the AWS ecosystem.
- ✓ Create a scalable and secure foundation for applications.
- ✓ Learn real-world infrastructure design for containerized workloads.

RESOURCES CREATED

- ✓ VPC with public and private subnets
- ✓ IAM roles and policies for ECS
- ✓ ECR repository for container images
- ✓ ECS Fargate cluster and service
- ✓ Application Load Balancer
- ✓ Security groups and target groups
- ✓ CloudWatch log group
- ✓ Required networking and routing

EXAMPLE: ECS CLUSTER (TERRAFORM)

```
resource "aws_ecs_cluster" "main" {
  name = "veriqta-ecs-cluster"

  setting {
    name = "containerInsights"
    value = "enabled"
  }

  tags = {
    Environment = "dev"
    Project     = "veriqta-container"
  }
}
```

This creates an ECS cluster with Container Insights enabled for better observability.

EXAMPLE: ECS SERVICE (FARGATE)

```
resource "aws_ecs_service" "app" {
  name             = "veriqta-app"
  cluster          = aws_ecs_cluster.main.id
  task_definition  = aws_ecs_task_definition.app.arn
  desired_count    = 2
  launch_type      = "FARGATE"

  network_configuration {
    subnets         = var.private_subnets
    security_groups  = [aws_security_group.ecs.id]
    assign_public_ip = false
  }
}
```

This runs a containerized application on ECS Fargate in private subnets (no public IP).

COMMON MISTAKES

- ✗ Running containers in public subnets
- ✗ Over-permissive IAM roles
- ✗ Not configuring logging and monitoring
- ✗ Exposing ECR repositories publicly
- ✗ Missing proper security group rules
- ✗ Not using private subnets for tasks
- ✗ Ignoring cost and scaling settings

JUNIOR ENGINEER TIPS

- ✓ Start with a simple 2-task service.
- ✓ Use private subnets for ECS tasks.
- ✓ Enable Container Insights.
- ✓ Use IAM roles, not access keys.
- ✓ Set proper logging and alarms.
- ✓ Test the full deployment end-to-end.
- ✓ Tag all resources.

REAL-WORLD IMPACT

- ✓ Provides a scalable and managed container platform.
- ✓ Reduces infrastructure overhead with Fargate.
- ✓ Supports modern application deployment workflows.
- ✓ Integrates with CI/CD pipelines.
- ✓ Builds a strong foundation for microservices and cloud-native apps.

VERIQTA

LEARN | BUILD | GROW



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

A Better
Engineer
Every Day

Build an EKS Infrastructure Foundation



Provision the infrastructure foundation for Amazon EKS, including networking, cluster configuration, worker capacity, IAM, and essential dependencies. Clearly separate Terraform's responsibility from Kubernetes workload management.

KEY COMPONENTS



VPC

Network for EKS cluster (public and private subnets).



Subnets

Private subnets for worker nodes, public subnets for load balancers.



IAM

Roles and policies for EKS and worker nodes.



EKS Cluster

Control plane, version, and essential settings.



Worker Capacity

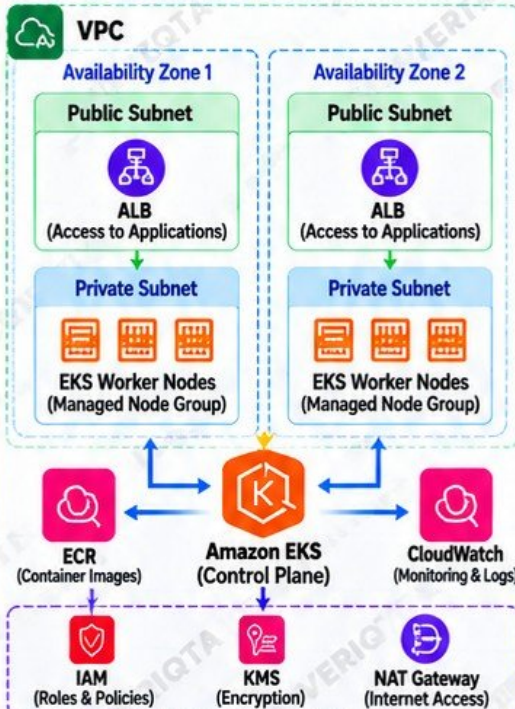
Managed node groups or self-managed nodes.



Supporting Services

Security groups, route tables, NAT, ECR (optional), CloudWatch, and KMS.

ARCHITECTURE DIAGRAM



TERRAFORM vs KUBERNETES



TERRAFORM MANAGES (THE INFRASTRUCTURE)

- ✓ VPC, subnets, route tables, NAT
- ✓ EKS cluster creation and configuration
- ✓ Worker node groups / node roles
- ✓ IAM roles and policies
- ✓ Security groups and load balancers
- ✓ Supporting AWS services (ECR, CloudWatch, KMS, etc.)



KUBERNETES MANAGES (THE WORKLOADS)

- ✓ Deploying applications (Deployments)
- ✓ Managing pods, services and ingresses
- ✓ Application configuration (ConfigMaps, Secrets)
- ✓ Scaling and self-healing
- ✓ Workload updates and rollbacks
- ✓ Application-level monitoring

IMPORTANT



Terraform builds the platform. Kubernetes runs the applications.

EXAMPLE: EKS CLUSTER (TERRAFORM)

```
resource "aws_eks_cluster" "main" {
  name     = "veriqta-eks"
  version  = "1.29"
  role_arn = aws_iam_role.eks_cluster.arn

  vpc_config {
    subnet_ids         = var.private_subnets
    endpoint_public_access = false
    endpoint_private_access = true
  }

  tags = {
    Environment = "dev"
    Project      = "veriqta-eks"
  }
}
```

Creates the EKS control plane with private endpoint access for better security.

JUNIOR ENGINEER TIPS



- ✓ Start with a small cluster for learning.
- ✓ Use private subnets for worker nodes.
- ✓ Enable cluster logging.
- ✓ Use managed node groups.
- ✓ Follow least-privilege IAM.
- ✓ Tag all resources.
- ✓ Understand the cost before scaling.
- ✓ Separate environments (dev, staging, prod).

COMMON MISTAKES



- ✗ Exposing the EKS API publicly without need.
- ✗ Using overly permissive IAM roles.
- ✗ Launching worker nodes in public subnets.
- ✗ Not enabling cluster logging.
- ✗ Ignoring security group rules.
- ✗ Not planning for regional capacity limits.
- ✗ Mixing infrastructure and application deployment in Terraform.

REAL-WORLD IMPACT



- ✓ Provides a secure and scalable Kubernetes platform.
- ✓ Supports modern application delivery.
- ✓ Enables team self-service and faster innovation.
- ✓ Reduces manual setup and configuration.
- ✓ Builds a strong foundation for production workloads.

USEFUL RESOURCES

- 🔗 <https://docs.aws.amazon.com/eks/>
- 🔗 <https://registry.terraform.io/providers/hashicorp/aws>
- 🔗 <https://docs.aws.amazon.com/prescriptive-guidance/latest/terraform-eks/>
- 🔗 <https://github.com/terraform-aws-modules/terraform-aws-eks>

VERIQTA

LEARN | BUILD | GROW



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

A Better
Engineer
Every Day

Troubleshoot a Broken Terraform Deployment



Intentionally investigate realistic failures such as dependency errors, incorrect variables, state problems, provider authentication failures, security-group mistakes, resource drift, and partially completed applies.

COMMON FAILURES AND FIXES

	Dependency Errors Resources created in the wrong order or missing dependencies.	→ Use <code>depends_on</code> , correct references, and check resource order in your configuration.
	Incorrect Variables Missing, wrong, or empty variable values.	→ Check <code>terraform.tfvars</code> , variable types, and use validation.
	State Problems Corrupted, missing, or locked state file.	→ Check backend configuration, unlock state, use <code>terraform state</code> commands.
	Provider Authentication Failures Invalid or expired AWS credentials.	→ Verify AWS credentials, profile, and permissions. Run <code>aws sts get-caller-identity</code> .
	Security-Group Mistakes Blocked traffic or wrong rules.	→ Review ingress/egress rules, ports, and source CIDR ranges.
	Resource Drift Manual changes outside Terraform.	→ Run <code>terraform plan</code> , use <code>terraform import</code> , and standardize changes through code.
	Partially Completed Applies Some resources created, others failed.	→ Identify what was created, fix the issue, and re-run <code>apply</code> . Use <code>terraform state</code> to clean up if needed.

USEFUL DIAGNOSTIC COMMANDS

<code>terraform init</code>	→ Initialize the project
<code>terraform validate</code>	→ Check configuration syntax
<code>terraform plan</code>	→ See what will change
<code>terraform apply</code>	→ Deploy the infrastructure
<code>terraform state list</code>	→ List resources in state
<code>terraform state show <resource></code>	→ Show resource details
<code>terraform state rm <resource></code>	→ Remove from state
<code>terraform import</code>	→ Import existing resources
<code>terraform refresh</code>	→ Sync state with real infrastructure
<code>terraform taint <resource></code>	→ Force resource recreation
<code>terraform untaint <resource></code>	→ Remove taint
<code>TF_LOG=DEBUG terraform apply</code>	→ Get detailed logs

EXAMPLE ERROR AND FIX

Error: Error creating EC2 instance: UnauthorizedOperation: You are not authorized to perform this operation.

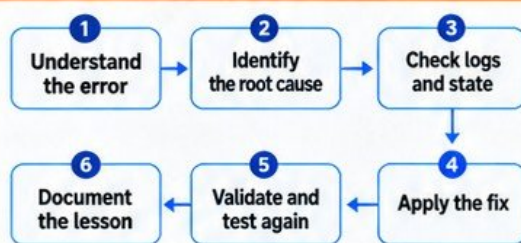
Possible Causes

- Missing IAM permissions
- Using the wrong AWS profile
- Organization SCP blocking the action

Fix Steps

- 1 Run `aws sts get-caller-identity` to verify your identity
- 2 Check your IAM policy permissions
- 3 Confirm you are using the correct AWS profile
- 4 Review organization service control policies (SCP)

TROUBLESHOOTING WORKFLOW



COMMON MISTAKES

- ✗ Ignoring error messages
- ✗ Not checking the Terraform logs
- ✗ Using hardcoded values
- ✗ Not understanding state
- ✗ Skipping terraform plan
- ✗ Making manual changes in the console
- ✗ Not using version control
- ✗ Not testing in a non-production environment

JUNIOR ENGINEER TIPS

- ✓ Read the full error message
- ✓ Use small, incremental changes
- ✓ Keep your state in remote backend
- ✓ Use descriptive variable names
- ✓ Always run `terraform plan`
- ✓ Verify AWS permissions early
- ✓ Check the actual resource in AWS
- ✓ Document what you learn

USEFUL RESOURCES

- 🔗 <https://developer.hashicorp.com/terraform/docs>
- 🔗 <https://registry.terraform.io/providers/hashicorp/aws/latest/docs>
- 🔗 <https://docs.aws.amazon.com/>
- 🔗 <https://spacelift.io/blog/terraform-errors>

REAL-WORLD IMPACT

- 🎯 Helps you diagnose and fix real production issues
- ✓ Reduces deployment downtime
- ✓ Builds confidence in infrastructure operations
- ✓ Prepares you for real DevOps and SRE roles

Capstone: Production Multi-Environment AWS Platform



Combine reusable modules, remote state, networking, load balancing, Auto Scaling, private databases, security, monitoring, CI/CD, and dev/staging/prod separation into one portfolio-grade Terraform project.

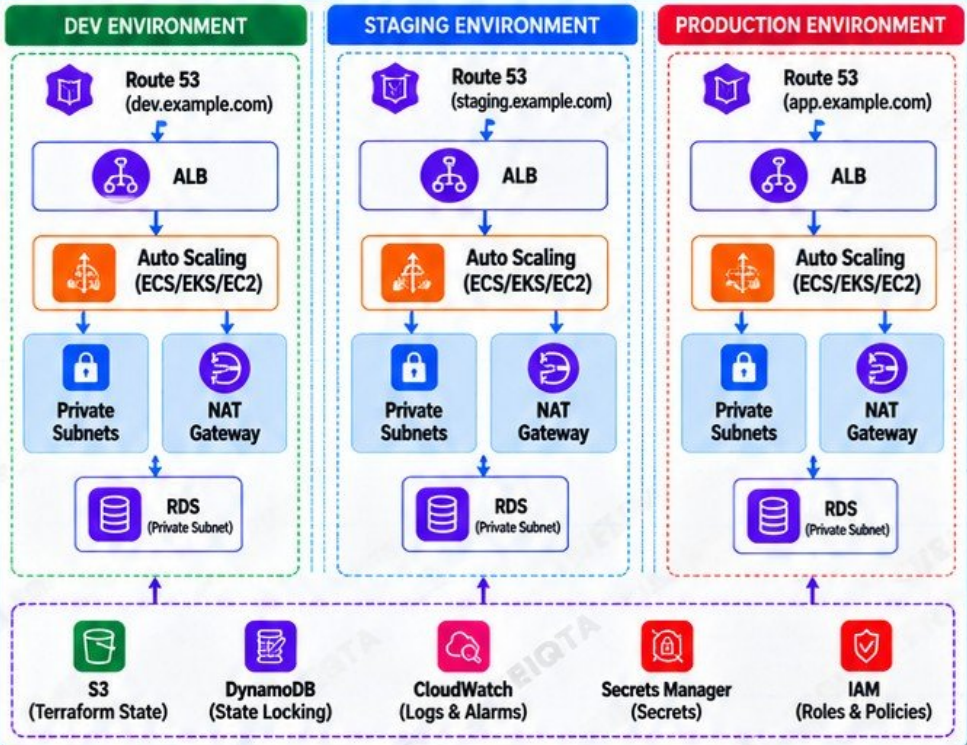
PROJECT GOALS

- ✓ Build a real-world, production-grade AWS platform with Terraform.
- ✓ Use reusable and scalable modules.
- ✓ Implement dev, staging, and production environments.
- ✓ Apply infrastructure security best practices.
- ✓ Enable monitoring, logging, and alerting.
- ✓ Automate deployments with CI/CD.
- ✓ Create a portfolio project ready for your GitHub.

KEY AWS SERVICES

-  **VPC**
Networking (public & private subnets)
-  **ALB**
Application Load Balancer
-  **Auto Scaling**
Scalable compute layer
-  **ECS / EKS**
Container platform (optional)
-  **RDS**
Private database layer
-  **S3**
Remote state and artifacts
-  **IAM**
Roles, policies, least privilege
-  **CloudWatch**
Monitoring, logs and alarms
-  **Secrets Manager**
Manage sensitive data
-  **Route 53**
DNS and domain management

HIGH-LEVEL ARCHITECTURE (MULTI-ENVIRONMENT)



TERRAFORM PROJECT STRUCTURE

```
aws-platform/
├── modules/
│   ├── vpc/
│   ├── compute/
│   ├── database/
│   ├── security/
│   └── monitoring/
├── environments/
│   ├── dev/
│   ├── staging/
│   └── prod/
├── .github/ (CI/CD workflows)
├── backend/ (remote state config)
└── README.md
```

Use modules for reusability, separate environments for isolation, and remote state for collaboration.

CI/CD WORKFLOW (GITHUB ACTIONS)



BEST PRACTICES

- ✓ Use remote state with S3 and DynamoDB.
- ✓ Keep environments isolated (dev, staging, prod).
- ✓ Use reusable, versioned modules.
- ✓ Follow least-privilege IAM.
- ✓ Enable encryption for all sensitive data.
- ✓ Implement monitoring and alerting.
- ✓ Automate with GitHub Actions.
- ✓ Tag all resources consistently.

REAL-WORLD IMPACT

- ✓ Demonstrates end-to-end infrastructure skills.
- ✓ Shows production readiness and security awareness.
- ✓ Builds a strong portfolio project for job opportunities.
- ✓ Teaches real DevOps and platform engineering practices.

LEARNING OUTCOMES

- ✓ Design and deploy a complete AWS platform.
- ✓ Manage multiple environments efficiently.
- ✓ Apply security, monitoring, and operational best practices.
- ✓ Gain hands-on experience with real-world Terraform workflows.

NEXT STEPS

- ✓ Add application deployment (ECS/EKS).
- ✓ Integrate with observability tools (Prometheus/Grafana).
- ✓ Implement disaster recovery (multi-region).
- ✓ Continuously improve and document your project.